Action, Func & Task

Delegate types, return types, and why `async void` exists at all

1. Two Different Categories

The confusion behind most async questions is treating these three as siblings. They aren't:

	What it is	Answers the question
`Action`	A **delegate type**	"What shape of method do you want?"
`Func<T>`	A **delegate type**	"What shape of method do you want?"
`Task`	A **class** (a return type)	"What does that method hand back?"

A delegate is a variable that holds a method. `Action` holds a method returning nothing; `Func<T>` holds one returning `T`.

```
Action greet = () ⇒ Console.WriteLine("hi");    // returns nothing
Func<int> answer = () ⇒ 42;                     // returns an int
Func<int, string> name = id ⇒ $"user-{id}";     // int in, string out
```

> **READ IT LIKE ENGLISH**
>
> In `Func<A, B, C>` the **last** type argument is always the return type; everything before it is a parameter. So `Func<int, string>` reads "takes an int, returns a string." `Action<int, string>` has no return type at the end — every argument is a parameter.

`Task` is not in that family at all. It's an object representing *work that is under way* — a handle you hold so you can later ask "finished yet?", "did it throw?", or "wait for it".

2. Task Is the Async Version of void

One substitution unlocks everything else:

```
synchronous          asynchronous
─────────────        ──────────────
   void        →        Task
    T          →      Task<T>
```

A synchronous method that returns nothing becomes an async method returning `Task`. One returning `int` becomes one returning `Task<int>`.

```
void Save() { }                              // nothing
async Task SaveAsync() { }                   // nothing, eventually

int Count() => 42;                          // an int
async Task<int> CountAsync() => 42;          // an int, eventually
```

> **METAPHOR**
>
> A `Task` is the cloakroom ticket. You hand over your coat (start the work) and get a ticket back immediately — you don't stand there holding the coat. Later you present the ticket to collect the result, or to learn that something went wrong. `Task<T>` is a ticket for a coat; plain `Task` is a ticket for "the job is done" with nothing to collect.

3. The Async Delegate Table

Apply that substitution to the delegate shapes and the whole table falls out:

Synchronous	Asynchronous	Reads as
`Action`	`Func<Task>`	no args, returns a Task
`Action<int>`	`Func<int, Task>`	int in, returns a Task
`Func<int>`	`Func<Task<int>>`	no args, returns a Task of int
`Func<int, string>`	`Func<int, Task<string>>`	int in, returns a Task of string

```
Action         sync  = ()       => Console.WriteLine("hi");
Func<Task>     async = async ()  => await Task.Delay(100);

Func<int>       syncValue  = ()       => 42;
Func<Task<int>> asyncValue = async ()  => { await Task.Delay(1); return 42; };
```

4. Why There Is No AsyncAction

People look for `AsyncAction` and can't find it. It cannot exist.

> **GOLDEN RULE**
>
> **"Async" means "returns a Task."** So an asynchronous method that "returns nothing" still returns something — the `Task` itself. That makes it a `Func<Task>`, never an `Action`. `Action` is defined by returning nothing at all, which an async method never does.

This is why the async counterpart of the *void*-shaped delegate is a *Func*-shaped one. The asymmetry looks odd until you see that `Task` occupies the return slot that `Action` leaves empty.

5. async void: What You Get When You Force It

C# does let you write an async lambda where an `Action` is expected. The compiler obliges by producing `async void` — a method that really is asynchronous but hands nothing back.

```
// The API only accepts Action:
token.Register(async () ⇒ await SeedDatabaseAsync());    // ⚠ async void
```

Three things follow, and all of them are bad:

- **Nobody can await it.** There is no `Task`, so the caller cannot wait for completion. Execution continues at the first `await` inside your lambda.
- **Exceptions are unobserved.** In an `async Task` method, a thrown exception is captured *into the Task* for the awaiter to see. With no Task, it is raised on the thread pool instead — which by default **crashes the process**.
- **It cannot be cancelled or tracked.** No handle means no cancellation, no continuation, no "is it done?"

> **COMMON CONFUSION**
>
> `async void` is not "async that returns nothing" — it is **async that returns no handle**. The work still happens; you have simply thrown away the only means of observing it. That is why the compiler warns, and why `try/catch` *inside* the lambda is the only thing standing between you and an unhandled crash.

The one legitimate use

Event handlers, where the framework's contract genuinely is fire-and-forget and the signature is fixed:

```
button.Click += async (sender, e) ⇒ await SaveAsync();    // ✅ acceptable
```

Even here, wrap the body in `try/catch` — there is nowhere else for a failure to surface.

6. Reading an API's Intent From Its Signature

The delegate type an API accepts tells you whether async work is welcome, before you write a line:

Signature	What the API is promising
`void Register(Action callback)`	"I'll call you and move on." No async work — it can't wait for you.
`void Use(Func<Task> handler)`	"I'll await your work." Async is expected.
`Task RunAsync(Func<CancellationToken, Task> work)`	"I'll await it and can cancel it."

Case study: CancellationToken.Register

`IHostApplicationLifetime.ApplicationStarted` is a `CancellationToken`, so registering a callback goes through `CancellationToken.Register`. Every overload is `Action`-shaped:

```
Register(Action)
Register(Action<object?>, object?)
Register(Action<object?, CancellationToken>, object?)
// Func<Task> — does not exist, deliberately
```

Token callbacks are meant to be short and synchronous, and nothing awaits them, so there is nowhere for a `Task` to go. Passing an async lambda gives you `async void` and a race between your startup work and incoming traffic.

The answer is not to find a workaround but to move the async work somewhere designed to hold it:

```
// A: await it before the host starts serving
await DbInitializer.InitDb(app);
app.Run();

// B: IHostedService — the framework's slot for async startup work
public class DbSeeder(IServiceProvider sp) : IHostedService
{
    public Task StartAsync(CancellationToken ct) ⇒ DbInitializer.InitDb(sp);
    public Task StopAsync(CancellationToken ct) ⇒ Task.CompletedTask;
}
```

Both are awaited, so failures stop startup instead of vanishing.

7. Key Takeaways

- **Different categories.** `Action` and `Func` are delegate types (a method's shape); `Task` is a return type (a handle to work in flight).
- **One substitution.** `void` → `Task`, `T` → `Task<T>`. Everything else follows.
- **`Action` → `Func<Task>`.** The async counterpart of a void-shaped delegate is Func-shaped, because the Task fills the empty return slot.
- **No `AsyncAction` is possible.** Async means returning a Task; returning nothing and returning a Task are contradictory.
- **`async void` returns no handle** — unawaitable, uncancellable, and its exceptions crash the process rather than landing in a Task. Acceptable only for event handlers, and even then wrap it in try/catch.
- **Read the signature.** `Action` means “I won't wait for you”; `Func<Task>` means “I will.” If an API only takes `Action`, your async work belongs elsewhere.